

Congestion and Curtailment Benefits Calculation Documentation

1 Introduction

The `congestion_curtailment_reduction.py` script calculates the benefits from reducing transmission congestion and renewable energy curtailment. It allocates effective capacity relief between curtailment and congestion to avoid double-counting, then monetizes both benefits over the project lifetime.

2 Method Overview

2.1 Purpose

This script calculates congestion and curtailment benefits by:

1. Determining effective capacity relief (greenfield vs. reconductoring)
2. Allocating capacity relief between curtailment and congestion
3. Calculating congestion reduction energy and costs
4. Calculating curtailment reduction energy and benefits
5. Applying saturation factors (haircuts) to account for diminishing returns
6. Computing residual congestion costs (unmet capacity needs)
7. Calculating present values of all benefits and costs

2.2 Calculation Approach

The calculation follows this structure:

Effective Capacity Relief:

$$\Delta C_{eff} = \begin{cases} \phi \times \text{Capacity} & \text{(Greenfield)} \\ \text{Capacity} - \text{Old Capacity} & \text{(Reconductoring)} \end{cases} \quad (1)$$

where ϕ is the flow factor.

Curtailment Benefits:

$$E_{curt} = H_c \times \min(\Delta C_{eff}, C_c) \quad (2)$$

$$\text{Benefit}_{curt} = E_{curt} \times \lambda_c \quad (3)$$

where H_c is curtailment hours, C_c is average curtailment MW, and λ_c is curtailment price.

Congestion Benefits:

$$E_{cong} = E_{cong,bc} + E_{cong,non} + E_{near} \quad (4)$$

$$\text{Benefit}_{cong} = E_{cong} \times \lambda_{cong} \times (1 - \text{saturation}) \quad (5)$$

where the energy is split between overlap hours (with curtailment) and non-overlap binding hours.

Residual Congestion Costs:

$$E_{residual} = H_{bc} \times \max(0, X - \Delta C_{remain,bc}) + H_{bnon} \times \max(0, X - \Delta C_{eff}) \quad (6)$$

$$\text{Cost}_{residual} = E_{residual} \times \lambda_{cong} \quad (7)$$

2.3 Inputs and Outputs

Inputs:

- Project technical details (capacity, old capacity, reconductoring status)
- Congestion parameters (binding hours, average exceedance, flow factor, saturation factor, congestion price)
- Curtailment parameters (curtailment hours, average curtailment MW, curtailment price, saturation factor)
- Financing parameters (WACC, timeline)

Outputs:

- Annual and lifetime congestion benefits (raw and haircut)
- Annual and lifetime curtailment benefits (raw and haircut)
- Residual congestion costs
- Congestion and curtailment costs during delay/construction (allocated, no double count)
- Present values of all metrics

2.4 Integration with Other Scripts

This script integrates with:

- `yaml_loaders.py` - Loads project configuration (see `utilities_documentation.tex`)
- `financial_utils.py` - Financial calculations (see `utilities_documentation.tex`)
- `csv_output_manager.py` - Writes results to CSV (see `csv_output_manager_documentation.tex`)

3 Key Functions

3.1 `allocate_curtailment_then_congestion(DC_eff_mw, ...)`

Allocates effective capacity relief between curtailment and congestion to avoid double-counting.

Parameters:

- `DC_eff_mw` (float): Effective capacity relief in MW
- `binding_hours_total` (float): Total binding constraint hours
- `average_exceedance_mw` (float): Average exceedance in MW
- `curtailment_hours_total` (float): Total curtailment hours
- `average_curtailment_mw` (float): Average curtailment in MW
- `average_curtailment_price` (float): Average curtailment price

Returns:

- Dictionary with curtailment benefits, overlap metrics, and remaining capacity for congestion

Key Logic:

```
1 # Curtailment relief (MWh/yr)
2 E_curt = Hc * min(DC, Cc)
3 curt_benefit = E_curt * lambda_c
4
5 # Overlap proxy between binding and curtailment time
6 theta = min(1.0, Hc / Hb) if Hc > 0 else 0.0
7 H_bc = theta * Hb # Overlap hours
8 H_bnon = (1.0 - theta) * Hb # Non-overlap hours
9
10 # Capacity used by curtailment in overlap
11 DC_used_bc = min(DC, Cc)
12 DC_remain_bc = max(0.0, DC - DC_used_bc)
```

3.2 calculate_congestion_reduction_costs(...)

Main function that calculates congestion and curtailment benefits.

Parameters:

- Multiple parameters including reconductoring status, capacities, congestion/curtailment parameters, financing parameters

Returns:

- Dictionary with all calculated benefits, costs, and present values

Key Logic:

```
1 # Determine effective capacity relief
2 if not reconductoring:
3     effective_capacity_relief = flow_factor * capacity_mw
4 else:
5     effective_capacity_relief = capacity_mw - old_capacity_mw
6
7 # Allocate between curtailment and congestion
8 alloc = allocate_curtailment_then_congestion(...)
9
10 # Congestion relief energy
11 E_cong_bc = H_bc * min(DC_remain, average_exceedance)
12 E_cong_non = H_bnon * min(effective_capacity_relief,
13     average_exceedance)
14 E_near = k * near_binding_hours * effective_capacity_relief
15 energy_congestion_reduction = E_cong_bc + E_cong_non + E_near
16
17 # Monetize with saturation haircut
18 annual_congestion_benefit_raw = energy_congestion_reduction *
19     congestion_price
20 annual_congestion_benefit_haircut = (1 - saturation_factor) *
21     annual_congestion_benefit_raw
22
23 # Residual congestion
24 energy_congestion_residual = H_bc * max(0, average_exceedance -
25     DC_remain)
26     + H_bnon * max(0, average_exceedance -
27     effective_capacity_relief)
28 annual_congestion_residual_cost = energy_congestion_residual *
29     congestion_price
```

3.3 main()

Main execution function that orchestrates the calculation and output.

4 Example

The following example demonstrates a typical usage:

```
1 # Example: 500 MW Greenfield Overhead AC line
2 #
3 # Inputs (from YAML):
4 #   - Capacity: 500 MW
5 #   - Flow factor: 0.7
6 #   - Binding hours: 2000 hours/year
7 #   - Average exceedance: 300 MW
8 #   - Curtailment hours: 1500 hours/year
9 #   - Average curtailment: 200 MW
10 #   - Congestion price: $50/MWh
11 #   - Curtailment price: $30/MWh
12 #   - Saturation factor: 0.2 (20% haircut)
13 #   - Project lifetime: 50 years
14 #
15 # Calculation:
16 #   Effective capacity relief = 0.7 * 500 = 350 MW
17 #
18 #   Curtailment:
19 #     E_curt = 1500 * min(350, 200) = 300,000 MWh/year
20 #     Benefit = 300,000 * $30 = $9M/year
21 #
22 #   Congestion (after curtailment allocation):
23 #     Overlap theta = min(1.0, 1500/2000) = 0.75
24 #     H_bc = 0.75 * 2000 = 1500 hours
25 #     H_bnon = 0.25 * 2000 = 500 hours
26 #     DC_remain = 350 - 200 = 150 MW
27 #     E_cong_bc = 1500 * min(150, 300) = 225,000 MWh/year
28 #     E_cong_non = 500 * min(350, 300) = 150,000 MWh/year
29 #     E_cong = 375,000 MWh/year
30 #     Benefit_raw = 375,000 * $50 = $18.75M/year
31 #     Benefit_haircut = (1 - 0.2) * $18.75M = $15M/year
32 #
33 #   Residual congestion:
34 #     E_residual = 1500 * max(0, 300-150) + 500 * max(0, 300-350)
35 #                 = 225,000 MWh/year
36 #     Cost = 225,000 * $50 = $11.25M/year
37 #
38 # Output:
39 #   - Annual curtailment benefit: $9M
40 #   - Annual congestion benefit (haircut): $15M
41 #   - Annual residual congestion cost: $11.25M
42 #   - Net annual benefit: $12.75M
```

Integration Example:

```

1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 results = {
4     "congestion_benefit_annual": annual_congestion_benefit_raw,
5     "congestion_benefit_pv": lifetime_congestion_benefit_pv,
6     "curtailment_benefit_annual": annual_curtailment_benefit,
7     "curtailment_benefit_pv": lifetime_curtailment_benefit_pv,
8     "residual_congestion_cost_annual":
9         annual_congestion_residual_cost,
10    # ... more fields
11 }
12 csv_manager.add_congestion_curtailment(results)
13 csv_manager.write_batch_summary()

```

5 Key Implementation Details

5.1 Double-Counting Avoidance

The script prevents double-counting by allocating capacity relief first to curtailment, then to congestion. The overlap factor (θ) accounts for hours when both curtailment and congestion occur. This same allocation is used for delay-period opportunity costs.

5.2 Saturation Factors

Saturation factors (haircuts) account for diminishing returns as more capacity is added. They reduce the monetized benefits to reflect that not all theoretical benefits are realized.

5.3 Residual Congestion

The script calculates residual congestion costs, representing unmet capacity needs when effective capacity relief is less than average exceedance. This is important for partial solutions (e.g., reconductoring that doesn't fully solve congestion).

5.4 Flow Factor

For greenfield projects, the flow factor (ϕ) represents the fraction of nameplate capacity that effectively relieves congestion, accounting for network constraints and power flow physics.

5.5 Near-Binding Hours

The script includes near-binding hours with a relief factor to capture benefits from reducing near-constraint conditions, not just fully binding constraints.

5.6 Delay and Construction Period Costs

The script calculates delay-period opportunity costs using the same curtailment-first allocation as operations, so overlapping hours are not double counted. This represents the value of forgone relief while the project is delayed.